University of Southampton

COMP6237 Data Mining
Lecture 4: Embedding Data (Dimensionality Reduction II)

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

Lecturer (Assistant Professor) @ VLC of ECS

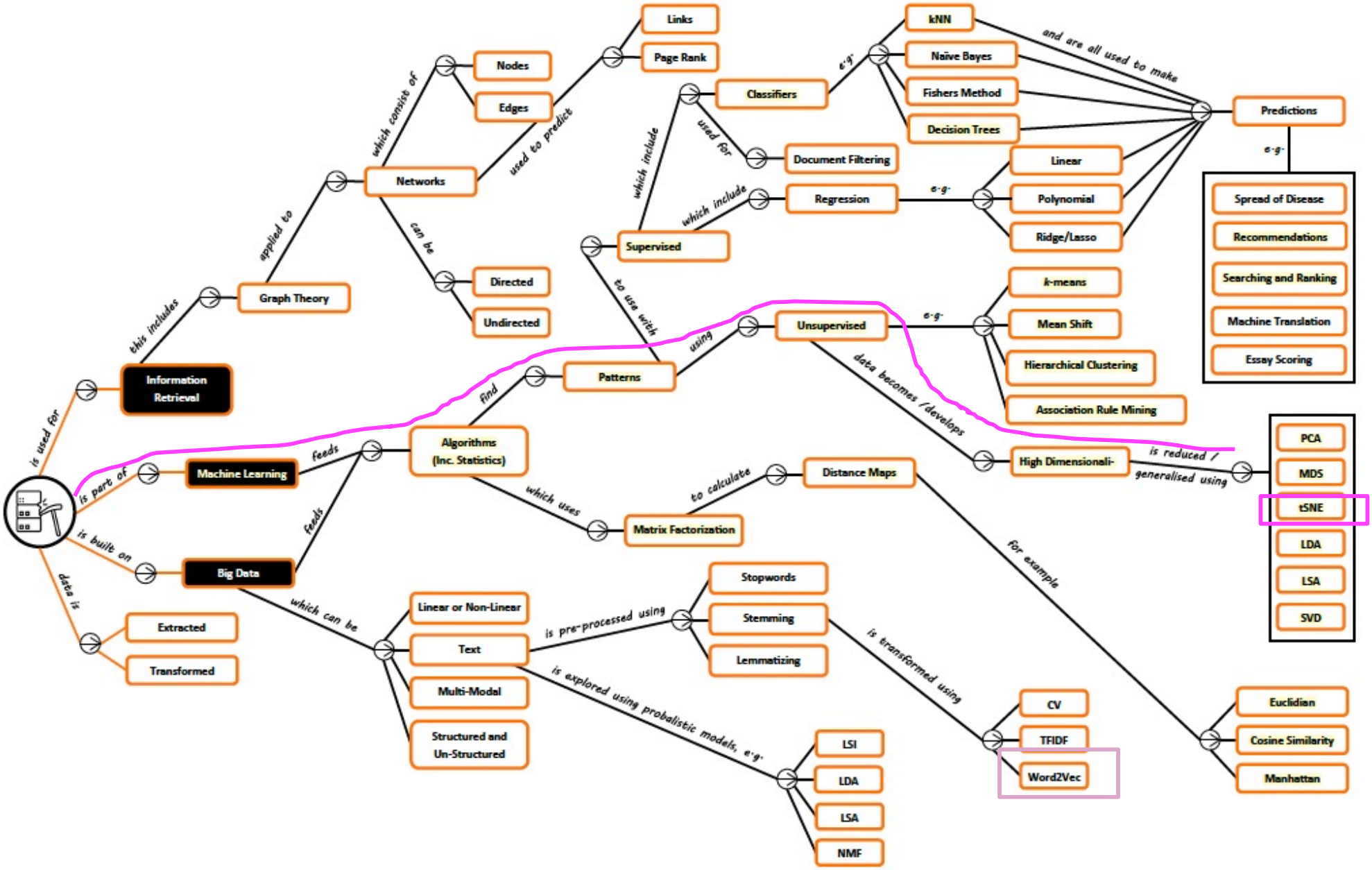
University of Southampton

Lecture slides available here:

http://comp6237.ecs.soton.ac.uk/zh.html

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

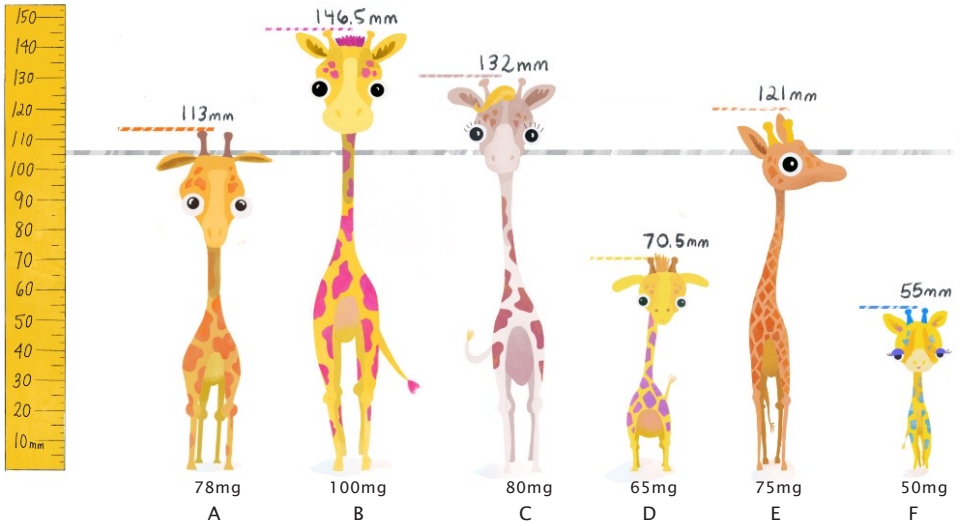
Dimensionality Reduction II – **Roadmap**



Dimensionality Reduction I (recap) - when data structure is simple and linear

University of Southampton

PCA works well here because the structure is simple and linear.
One direction explains most of the variation.
Projecting onto that line keeps most of the information.



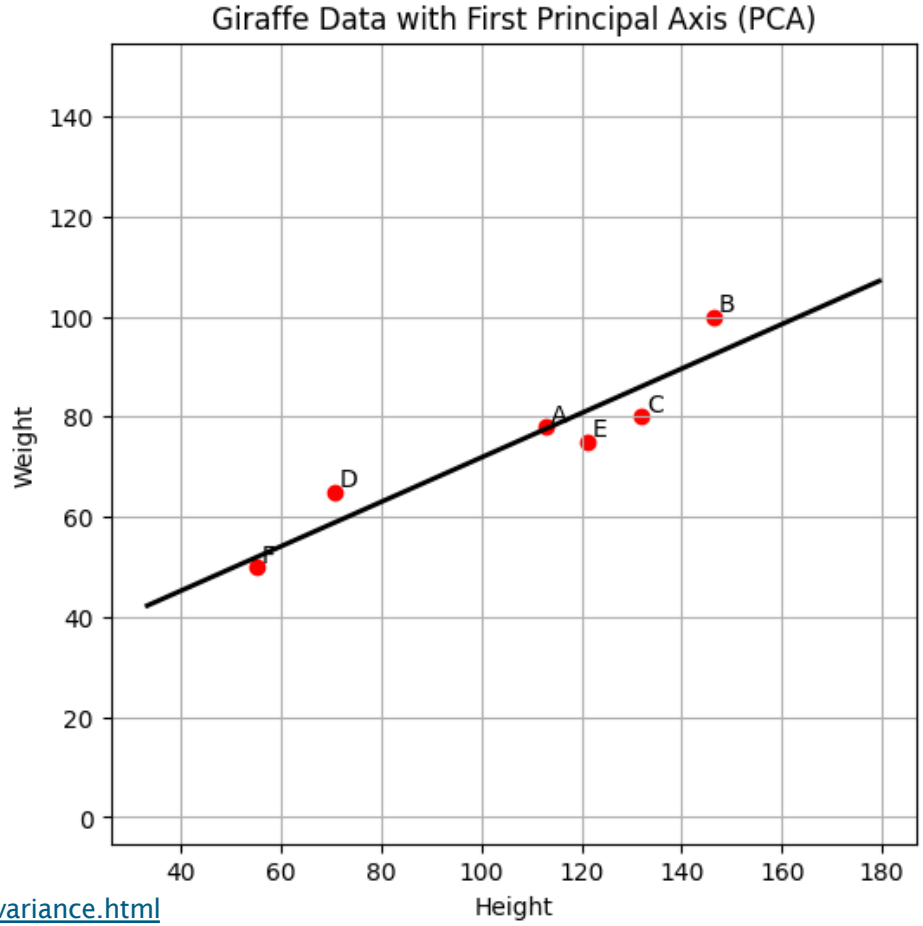


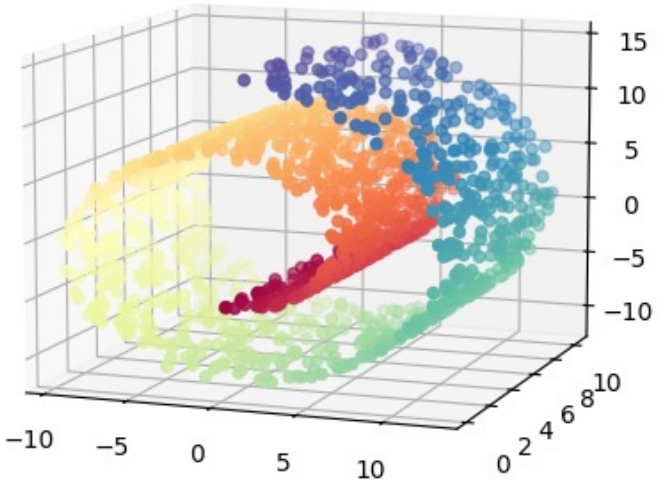
Image source: https://tinystats.github.io/teacups-giraffes-and-statistics/04_variance.html

Dimensionality Reduction II
- what if data structure gets more complex?

University of Southampton

But what if giraffes form groups not just by size?

- Maybe by habit, age, or health
- Some giraffes might be similar locally, even if they differ globally

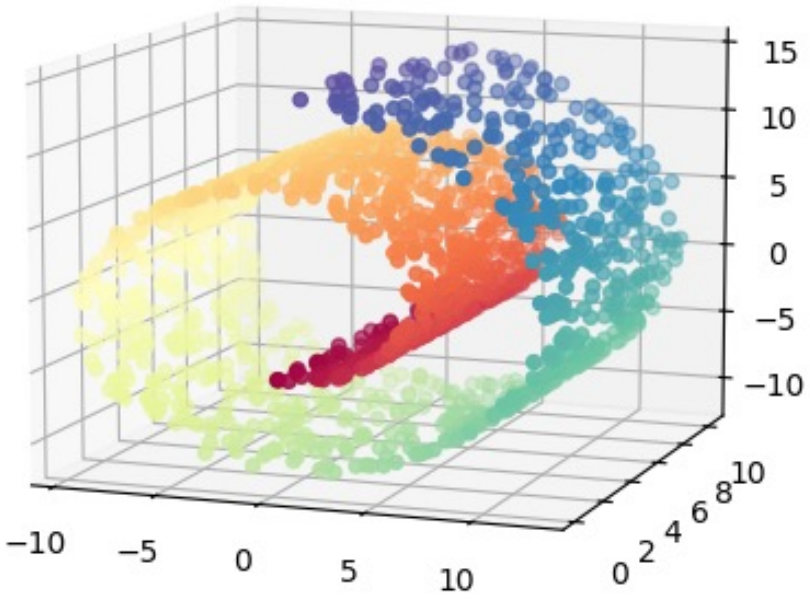


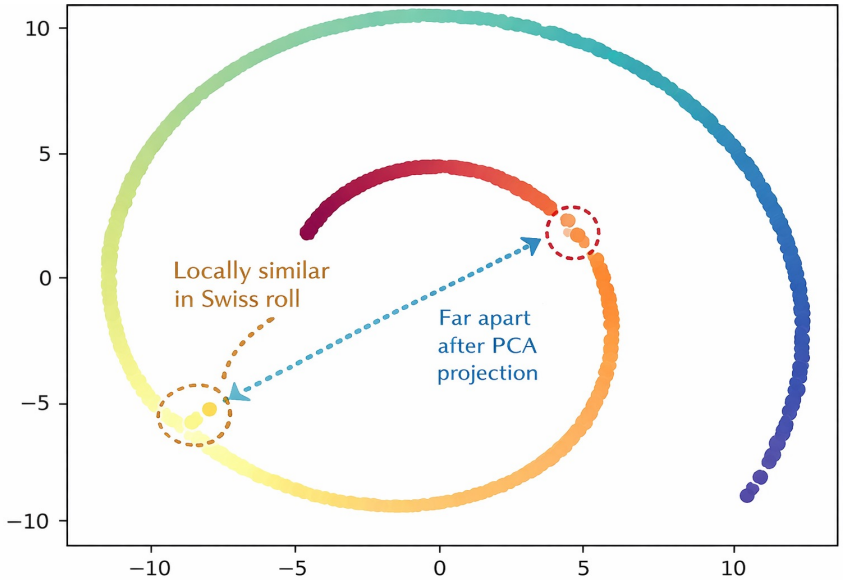
Swiss-roll data

Image source: ChatGPT

Dimensionality Reduction II
- what if data structure gets more complex?

University of Southampton





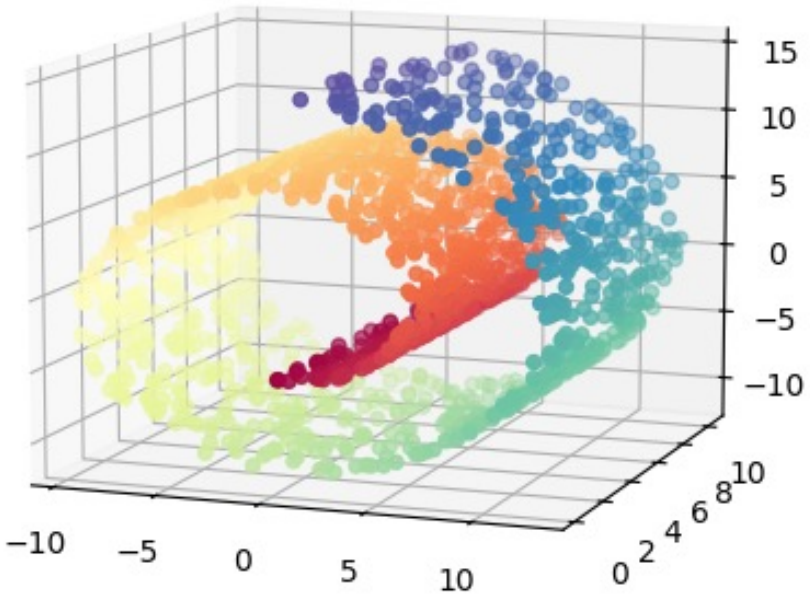
The Swiss roll is a **non-linear manifold.** Locally, points that are next to each other really are similar, but **globally the structure is curved and twisted.**

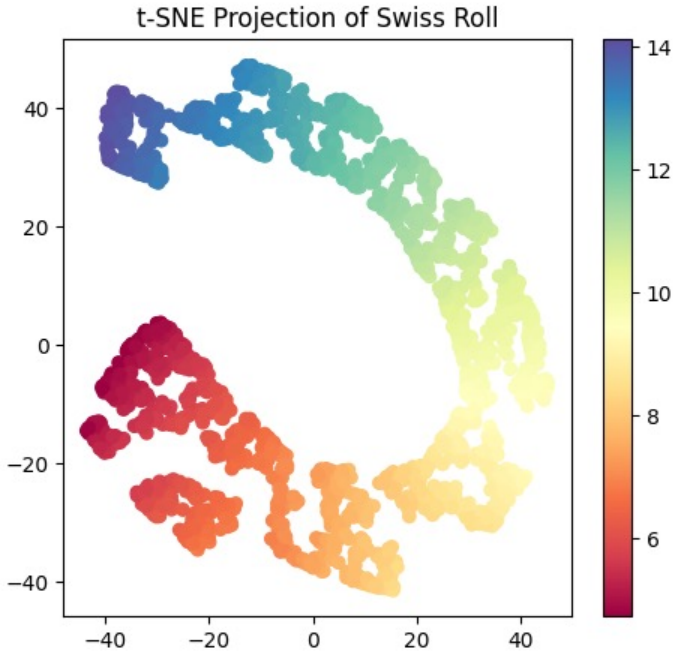
✓ **Captures global structure** by finding linear directions that maximize overall variance

✖ **Does not preserve local structure** (neighborhood relationships) on non-linear data

Dimensionality Reduction II
- what if data structure gets more complex?

University of Southampton



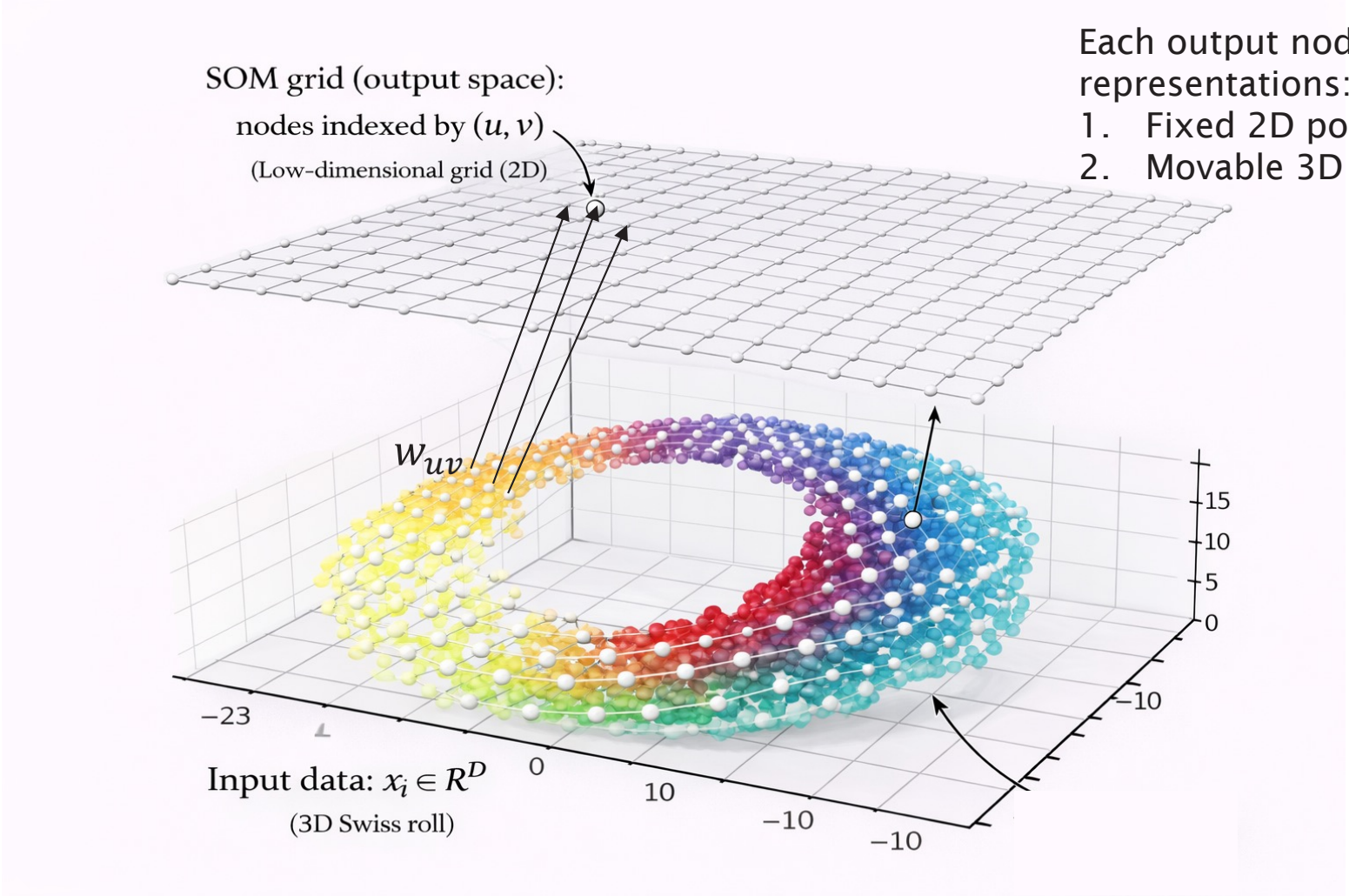


The Swiss roll is a **non-linear manifold.**
Locally, points that are next to each other really are similar,
but **globally the structure is curved and twisted.**

✓ **Separates clusters clearly in the embedding** (low-dim space) (helps reveal groups visually)

✓ **Preserves local neighborhood structure very well** (points that are close in the original space stay close in 2D)

Dimensionality Reduction II – Self-Organising Maps



Each output node has two representations:

1. Fixed 2D position (u, v)
2. Movable 3D weight vector w_{uv}

Idea: SOM keeps a fixed 2D grid, but move each node's weight vector in data space so the grid adapts to the data while preserving neighbourhoods.

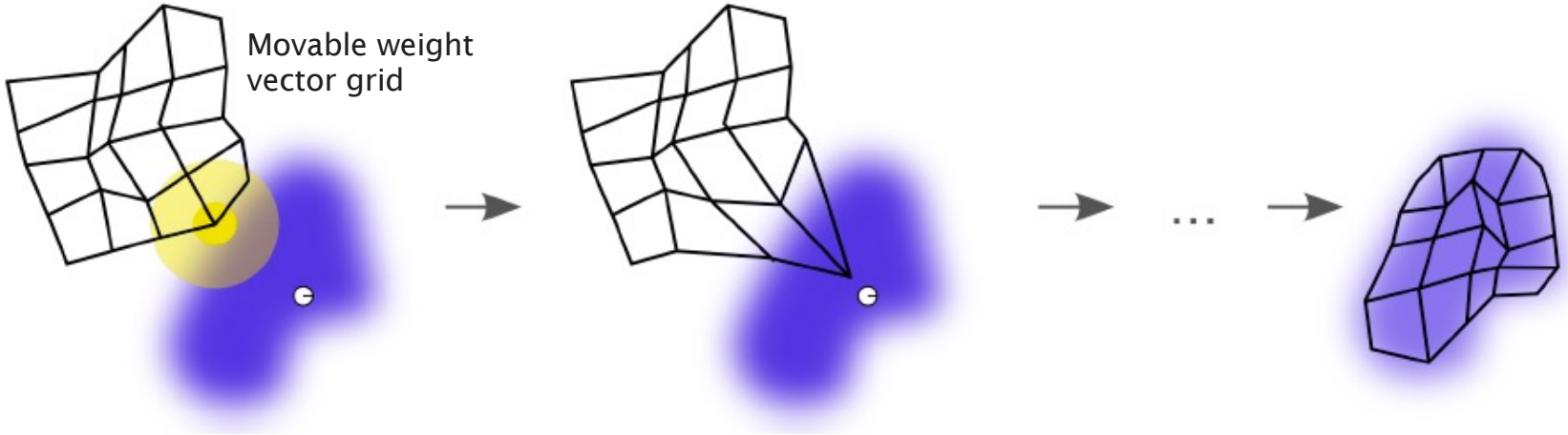
Dimensionality Reduction II – Self-Organising Maps

SOMs: two phases

- training
- mapping

To start training, the set of nodes each has a random starting position defined in the feature space.

This is then updated by taking one feature vector, finding which unit is the *best matching unit* (BMU) then moving that unit and, to a lesser extent, its neighbours, closer to that data point



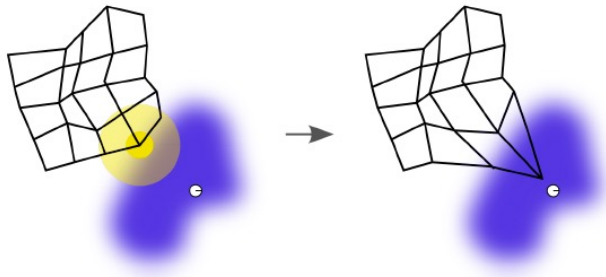
Dimensionality Reduction II – Self-Organising Maps

To update the weight vector $\boldsymbol{w}$

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) + \theta(u, v, t)\alpha(t)(x_i - \boldsymbol{w}(t))$$

where θ is the neighbourhood weighting function (usually Gaussian)

e.g., $\theta = \begin{cases} 1 & \text{if the node is the winner} \\ 0.5 & \text{if the node is immediate neighbour of the winner} \\ 0 & \text{otherwise} \end{cases}$



α is the learning rate

x_i is the input vector

u is the winner node, and v is the node whose weight is $\boldsymbol{w}$

Both the learning rate and the neighbourhood weighting function get smaller over time

Dimensionality Reduction II– Self-Organising Maps

Algorithm 1: Training Self-Organising Maps

Data: N data points with d dimensional feature vectors X_i
$i = 1 \ldots N$, number of iterations λ

$\boldsymbol{w}$ = randomly initialise n xm units with weight vector ;

t = 0;

while $t < \lambda$ **do**

  for *each* x_i **do**

    $BMU = w_{nm}$ with min distance;

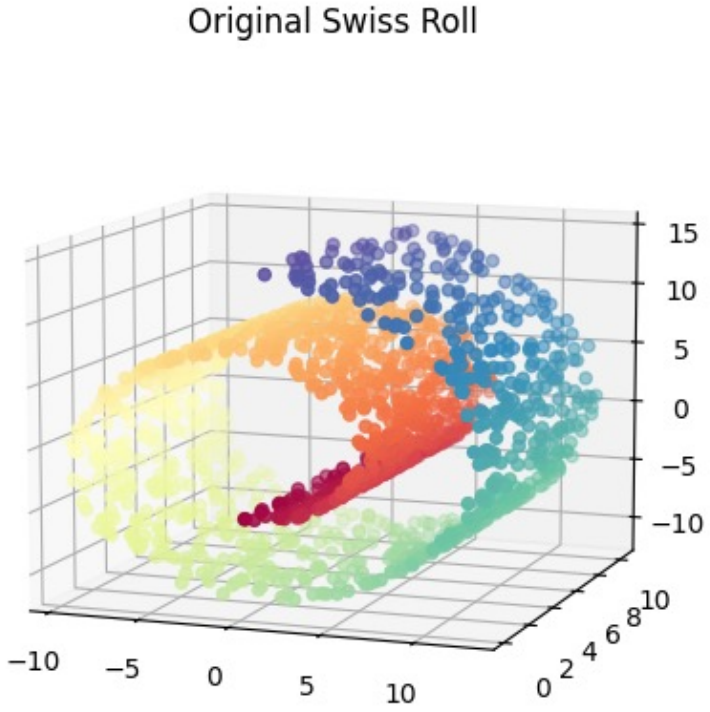
    Update BMU and its neighbours by moving closer to x_i;

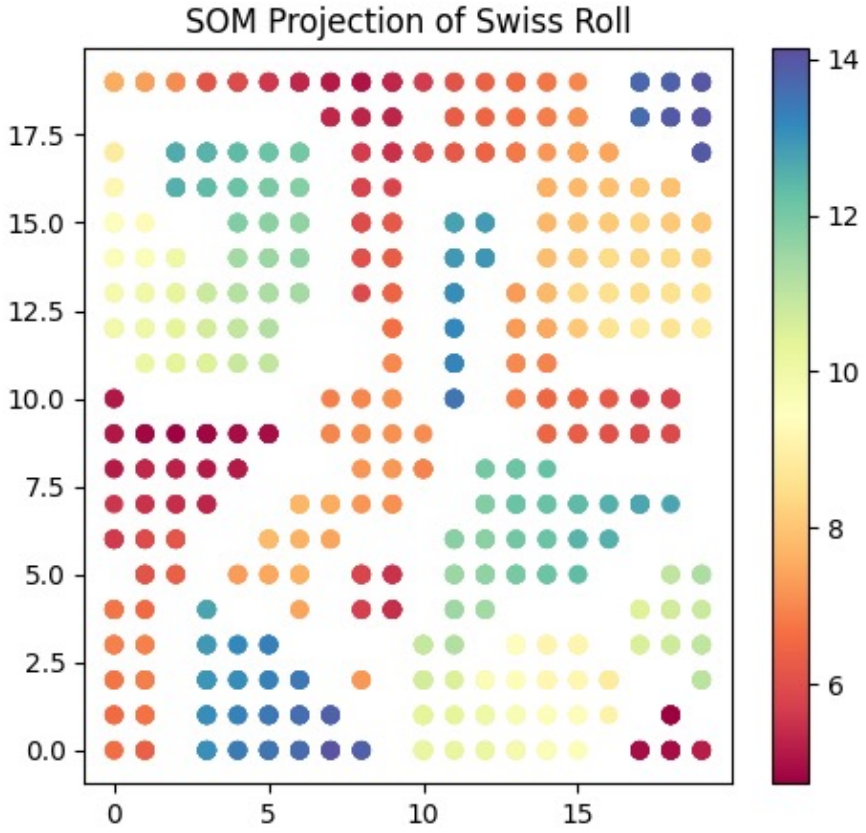
  end

  $t = t + 1$

end

Dimensionality Reduction II– Self-Organising Maps





The data in 3D has a clear structure

Topological structure is preserved, but group structure is not kept well?

ipynb mean centering demo https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19

Dimensionality Reduction II – Stochastic Neighbour Embedding

University of Southampton

Stochastic Neighbour Embedding (SNE)

SNE optimises the distribution of data

Aims to make the distribution of the projected data in low dimensional space close to the actual distribution in high dimensional space

Dimensionality Reduction II – Stochastic Neighbour Embedding

University of Southampton

To calculate the source distribution:

We define a conditional probability that high-dimensional x_i would pick x_j as a neighbour if the neighbours were picked in proportion to their probability density under a Gaussian centred at x_i

$$p_{j|i} = \frac{exp^{-||x_i - x_j||^2/2\sigma_i^2}}{\sum_{k \neq i} exp^{-||x_i - x_k||^2/2\sigma_i^2}}$$

The SNE algorithm chooses σ for each data point such that smaller σ is chosen for points in dense parts of the space, and larger σ is chosen for points in sparse parts

Dimensionality Reduction II – Stochastic Neighbour Embedding

University of Southampton

To calculate the target distribution:

Define a conditional probability that low-dimensional y_i would pick y_j as a neighbour if the neighbours were picked in proportion to their probability density under a Gaussian centred at y_i

$$q_{j|i} = \frac{exp(-||y_i - y_j||^2)}{\sum_{k \neq i} exp(-||y_i - y_k||^2)}$$

In this space we assume the variance of all Gaussians is $1/\sqrt{2}$ in this space

Dimensionality Reduction II – Stochastic Neighbour Embedding

University of Southampton

To measure the difference between two probability distributions we use the *Kullback-Leibler* (KL) Divergence:

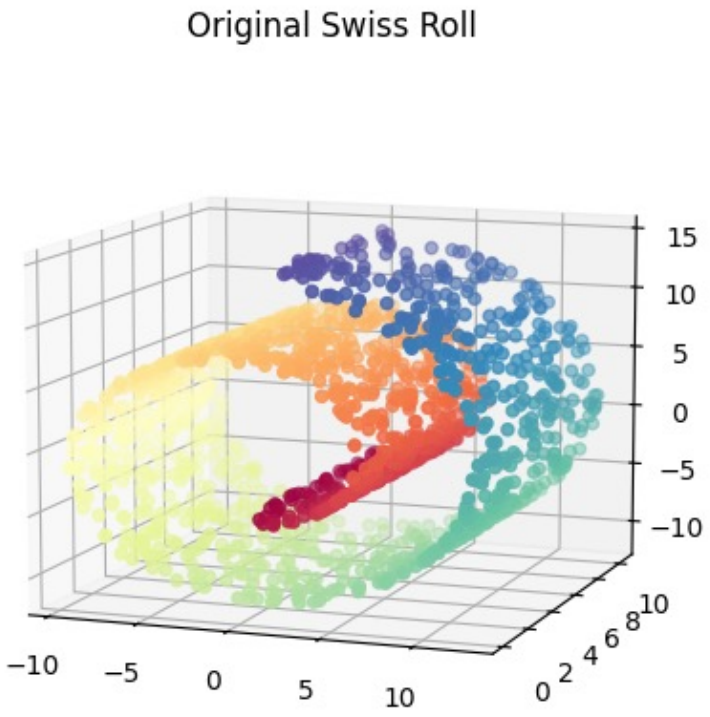
$$D_{KL}(P|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

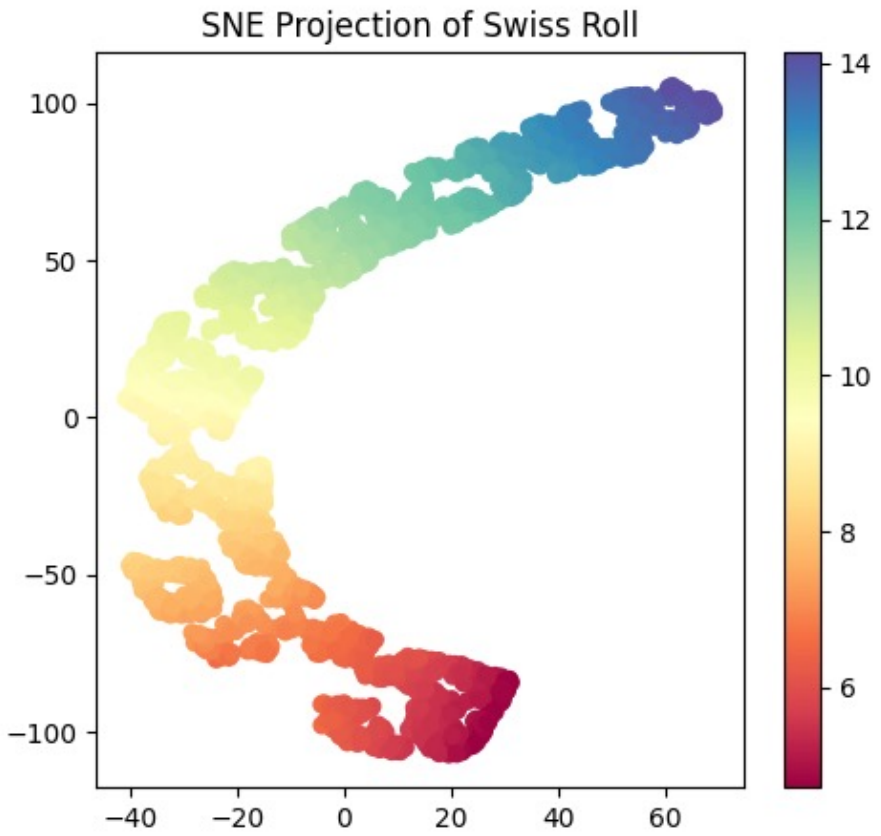
The cost function is the KL divergence summed over all data points

$$C = \sum_i \sum_j p_{i|j} \log \frac{p_{j|i}}{q_{j|i}}$$

C can be minimised using *gradient descent* - but..
difficult to optimise, leads to crowded visualisations, big clumps of data together in the center

Dimensionality Reduction II – Stochastic Neighbour Embedding

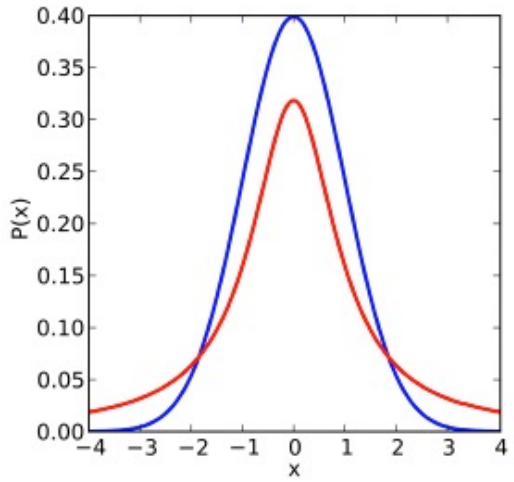




Dimensionality Reduction II – t- Stochastic Neighbour Embedding

University of Southampton

In 2008 Maaten and Hinton came up with a way to improve SNE, by replacing the Gaussian distribution for the lower dimensional space with a Student's t distribution.



Student's t distribution in red, Gaussian distribution in blue

The Student's t distribution has a much longer tail, helps avoid clumping in the middle.

Dimensionality Reduction II – t- Stochastic Neighbour Embedding

University of Southampton

The cost function is also modified, making gradients simpler, so faster to compute

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

To alleviate crowding using Student's t distribution in the lower dimensional space:

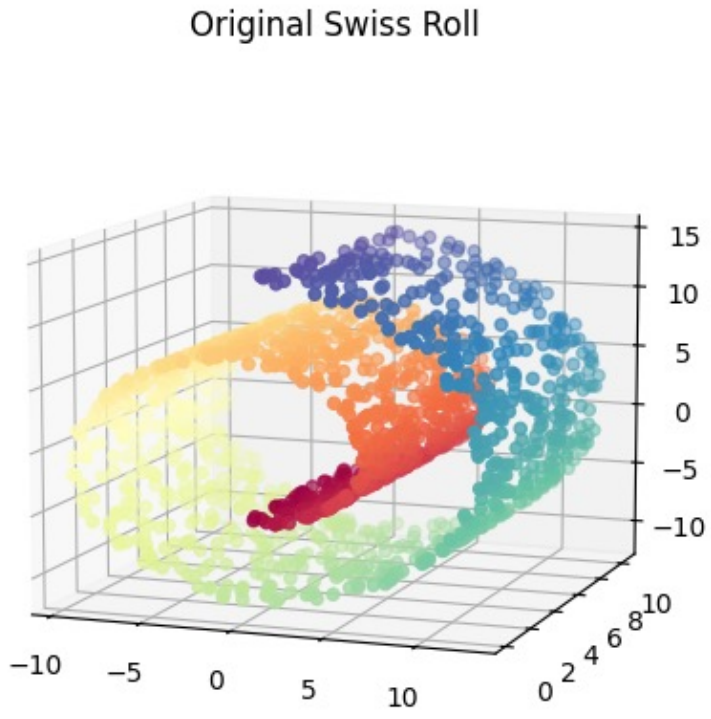
$$q_{ij} = \frac{(1 + ||x_i - x_j||)^{-1}}{\sum_{k \neq i}(1 + ||x_i - x_k||)^{-1}}$$

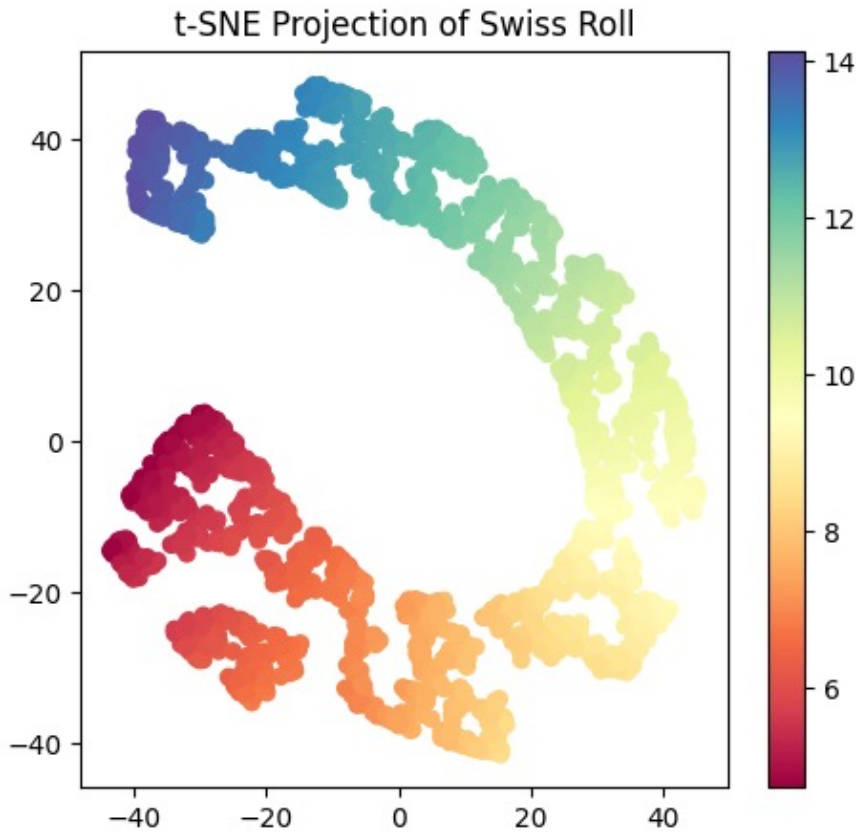
we use 1 degree of freedom with the Student's t distribution, equivalent to a Cauchy distribution

Dimensionality Reduction II – t- Stochastic Neighbour Embedding

University of Southampton

For the Swiss Roll data:





ipynb mean centering demo https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19

Dimensionality Reduction I & II– Summary

University of Southampton

Which Dimensionality Reduction Method Should I Use?

Your Goal	Data Structure	Recommended Method	Why
Reduce dimensions for speed or storage	Linear, simple	**PCA**	Preserves maximum variance, fast and stable
Remove noise before another algorithm	Mostly linear	**PCA**	Keeps main signal, removes minor variation
Visualise high-dimensional data	Any, especially complex	**t-SNE**	Excellent at revealing clusters
Preserve local neighbourhoods	Non-linear manifold	**SOM / t-SNE**	Keeps nearby points together
Understand topology and structure	Non-linear, continuous	**SOM**	Preserves topology explicitly
Discover clusters visually	Unknown structure	**t-SNE**	Strong cluster separation
Interpret global distances	Linear or near-linear	**PCA**	Distances remain meaningful
Teaching / intuitive explanation	Any	**SOM**	Very visual and conceptually clear

Dimensionality Reduction II – **Learning Outcomes**

- **LO1**: understand the basic idea of all the learned dimensionality reduction methods (exam)
  - E.g., describe the basic idea of the suggested algorithm
  - E.g., understand the pros and cons of the learned algorithms
  - E.g., given a dataset, be prepared to follow the selected algorithm to calculate its key steps on the given data
- **LO2**: Implement the learned algorithms for dimensionality reduction (course work)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf*
- *ChatGPT or other AI-based techs*

Dimensionality Reduction II – Textbook

University of Southampton

CHAPTER 7 Dimensionality Reduction

We saw in Chapter 6 that high-dimensional data has some peculiar characteristics, some of which are counterintuitive. For example, in high dimensions the center of the space is devoid of points, with most of the points being scattered along the surface of the space or in the corners. There is also an apparent proliferation of orthogonal axes. As a consequence high-dimensional data can cause problems for data mining and analysis, although in some cases high-dimensionality can help, for example, for nonlinear classification. Nevertheless, it is important to check whether the dimensionality can be reduced while preserving the essential properties of the full data matrix. This can aid data visualization as well as data mining. In this chapter we study methods that allow us to obtain optimal lower-dimensional projections of the data.

- Data Mining and Machine Learning: Fundamental Concepts and Algorithms M. L. Zaki and W. Meira

Chapter 11

Dimensionality Reduction

There are many sources of data that can be viewed as a large matrix. We saw in Chapter 5 how the Web can be represented as a transition matrix. In Chapter 9, the utility matrix was a point of focus. And in Chapter 10 we

- Mining of Massive Datasets J. Leskovec *et al*

Appendix

Dimensionality reduction and visualisation is key to understanding the data

Useful for your coursework

There are many ways to do this, with the `sklearn.manifold` library:

https://scikit-learn.org/stable/modules/manifold.html